

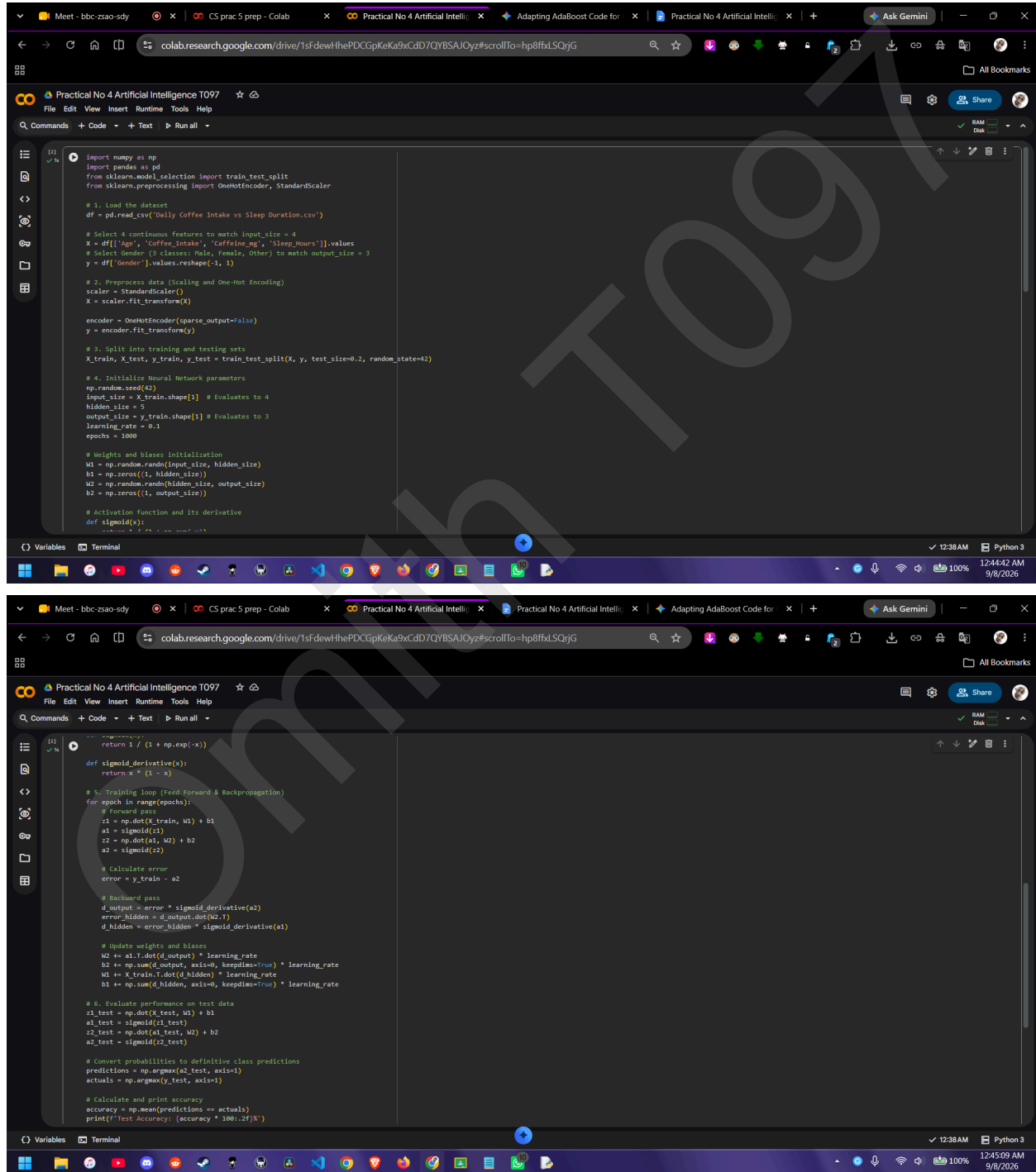
SHETH L.U.J & SIR M.V COLLEGE
Data Visualization with Power BI and Tableau

Practical No 4

Aim: Feed Forward Backpropagation Neural Network

- Implement the Feed Forward Backpropagation algorithm to train a neural network.
- Use a given dataset to train the neural network for a specific task.
- Evaluate the performance of the trained network on test data.

Code:



```
(1) import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler

# 1. Load the dataset
df = pd.read_csv('Daily Coffee Intake vs Sleep Duration.csv')

# 2. Select 4 continuous features to match input_size = 4
X = df[['Age', 'Coffee_intake', 'Caffeine_mg', 'Sleep_Hours']].values
# Select gender (3 classes: Male, Female, Other) to match output_size = 3
y = df['gender'].values.reshape(-1, 1)

# 3. Preprocess data (Scaling and One-Hot Encoding)
scaler = StandardScaler()
X = scaler.fit_transform(X)

encoder = OneHotEncoder(sparse_output=False)
y = encoder.fit_transform(y)

# 4. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# 5. Initialize Neural Network parameters
np.random.seed(42)
input_size = X_train.shape[1] # Evaluates to 4
hidden_size = 5
output_size = y_train.shape[1] # Evaluates to 3
learning_rate = 0.1
epochs = 1000

# Weights and biases initialization
W1 = np.random.randn(input_size, hidden_size)
b1 = np.zeros((1, hidden_size))
W2 = np.random.randn(hidden_size, output_size)
b2 = np.zeros((1, output_size))

# Activation function and its derivative
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

# 6. Training loop (Feed Forward & Backpropagation)
for epoch in range(epochs):
    # Forward pass
    z1 = np.dot(X_train, W1) + b1
    a1 = sigmoid(z1)
    z2 = np.dot(a1, W2) + b2
    a2 = sigmoid(z2)

    # Calculate error
    error = y_train - a2

    # Backward pass
    d_output = error * sigmoid_derivative(a2)
    error_hidden = d_output.dot(W2.T)
    d_hidden = error_hidden * sigmoid_derivative(a1)

    # Update weights and biases
    W2 += a1.T.dot(d_output) * learning_rate
    b2 += np.sum(d_output, axis=0, keepdims=True) * learning_rate
    W1 += X_train.T.dot(d_hidden) * learning_rate
    b1 += np.sum(d_hidden, axis=0, keepdims=True) * learning_rate

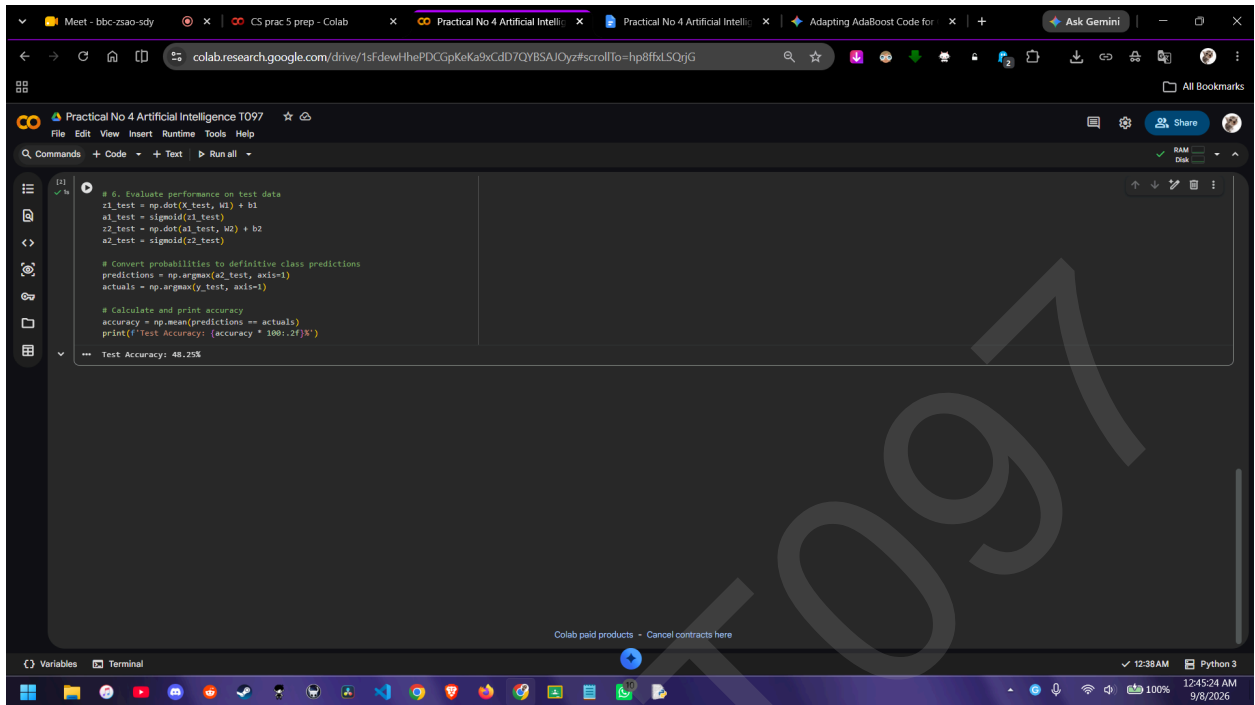
# 7. Evaluate performance on test data
z1_test = np.dot(X_test, W1) + b1
a1_test = sigmoid(z1_test)
z2_test = np.dot(a1_test, W2) + b2
a2_test = sigmoid(z2_test)

# Convert probabilities to definitive class predictions
predictions = np.argmax(a2_test, axis=1)
actuals = np.argmax(y_test, axis=1)

# Calculate and print accuracy
accuracy = np.mean(predictions == actuals)
print(f"Test Accuracy: {accuracy * 100:.2f}%")
```

SHETH L.U.J & SIR M.V COLLEGE
Data Visualization with Power BI and Tableau

Output:



The screenshot shows a Google Colab notebook titled "Practical No 4 Artificial Intelligence T097". The code in the notebook is as follows:

```
# 6. Evaluate performance on test data
z1_test = np.dot(X_test, W1) + b1
a1_test = sigmoid(z1_test)
z2_test = np.dot(a1_test, W2) + b2
a2_test = sigmoid(z2_test)

# Convert probabilities to definitive class predictions
predictions = np.argmax(a2_test, axis=1)
actuals = np.argmax(y_test, axis=1)

# Calculate and print accuracy
accuracy = np.mean(predictions == actuals)
print('Test Accuracy: (accuracy * 100:.2f)%')
```

The output of the code is displayed at the bottom of the cell: **Test Accuracy: 48.25%**.